Summary of "SuperBench: Improving Cloud AI Infrastructure Reliability with Proactive Validation"

Arnav Reddy (arnavr), Nishant Dash (ndash), Muzhe Wu (henrw), Yuxuan Liu (liurick)

Problem and Motivation

In a large-scale AI cloud infrastructure, consisting of hundreds of thousands of GPUs, devices fail regularly. To combat this, service providers have implemented hardware redundancies such as extra GPUs or network switches. However, the redundancies can mask some failures and degradation, leading to what the authors call “gray-failures” which impact the performance of AI workloads and conceal the root causes which may lead to failures. Additionally, when fixing an issue, service providers may make fixes until the system is performing as expected, but the redundancies can still be in a failed state. The authors attempt to mitigate the problem by proactively searching for failed nodes in the system through the use of the proactive validation system: SuperBench.

Related Works

Gray Failures:

- Fail-Stutter Fault Tolerance - Proposes a model to solve issue of the same components behaving differently
- Gray Failure: The Achilles’ Heel of Cloud-Scale Systems - Discusses gray failures and different observability traits in cloud systems

Elastic Training and Fault Tolerance

- Torch Elastic - Pytorch implementation of elastic training and fault-tolerant training

Performance Benchmarks

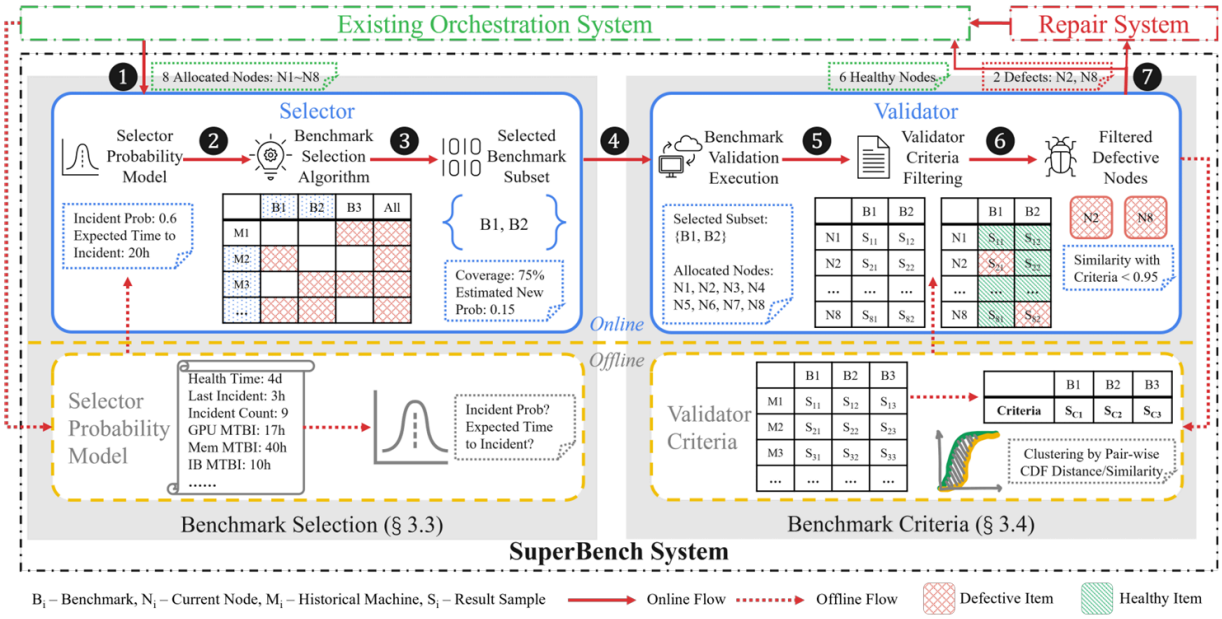
- DeepBench - Benchmarks typical operations in deep learning
- DAWNBench - Considers end-to-end training or inference performance and time to convergence

Solution Overview

The authors describe a two-part system involving a Selector, that will pick a subset of benchmarks and nodes to run those benchmarks on, and a Validator, that will evaluate the performance of the benchmarks and determine if a node is defective or not.

The offline Selector probability model is trained on the initial system deployment and is then used to determine which nodes are the closest to their next failure. The benchmark selection algorithm selects benchmarks that maximizes the probability of finding a defective node, while minimizing the runtime of the benchmarks.

The Validator first runs the micro benchmarks, which test individual components like the GPU compute units or device interconnects, and then runs the end-to-end benchmarks, which simulate a typical customer workload. The Validator uses criteria trained offline as well as an online algorithm to filter the defective nodes for repair.



Limitations

The authors mention that microbenchmarks can resolve the problem of having a limited workload suite, which obviously does not have complete coverage. However, despite microbenchmarks, new workloads must first cause incidents before they can be analyzed and added to the benchmark suite.

The SuperBench system also relies on the Selector's probability model which is purely based on large amounts of historical data. As the authors accurately mention: data center hardware is rapidly evolving and ultimately faces regressions. The SuperBench system may be less accurate when dealing with new data center clusters and hardware, which have less operational history.

Other factors such as non-deterministic workloads, the time and monetary costs of running a benchmark suite, and the binding of tests to specific cores and memory regions also pose limitations for SuperBench.

Additionally, the paper was only tested in Microsoft's Azure and not generalized.

Future Research Directions

- In order to resolve the issue of a limited workload suite, future research might explore the automatic generation of representative benchmarks given customer workloads.
- The incident probability model could also be improved beyond primarily considering hardware age to factoring in real-time data such as server-rack temperature to increase prediction accuracy.

Summary of Class Discussion

Q: Was the Cox time model proposed by the authors, or is it an existing method? What is special about the Cox model?
A: The Cox time model existed prior to the paper but it accepts more time-based inputs compared to alternatives which increases accuracy.

Q: Is SuperBench open-sourced? How do open source users generate benchmark failure probabilities?
A: Superbench is generalizable to allow third parties to add end-to-end tests / microbenchmarks. The probabilities for new tests come from initial test runs on a new, working system.

Q: How was the ideal line made (regarding cluster utilization)?
A: The authors designed the ideal line to simulate a world where there was no downtime from lost nodes.

Q: Was this used in the real world (since results are simulations)?
A: The solution was deployed in Microsoft Azure systems for 2+ years prior to the paper.

Q: 10.36% of nodes were classified as problematic. What does “problematic” mean? At what point are we confident enough to make a judgment?
A: “Problematic” nodes are those consistently flagged by benchmark anomalies and predictive models as likely gray failures. There is no absolute ground truth. Confidence is based on empirical thresholds. It can be deemed as a limitation.

Q: What are other errors that are not detected by SuperBench? Any formal definition of “gray failure”?
A: The collection of benchmarks is not exhaustive in capturing all hardware/software anomalies. It detects those that manifest benchmark degradations. However, SuperBench is extensible.

Summary of "Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks"

Arnav Reddy (arnavr), Nishant Dash (ndash), Muzhe Wu (henrw), Yuxuan Liu (liurick)

Problem and Motivation

Deep learning training pipelines are complex and frequently updated across many layers (user code, compilers, frameworks, drivers, distributed systems). Many failures don't crash jobs, but produce silent errors that surface late, wasting computing resources and obscuring root causes. Silent errors can't be detected easily from the noisy high-level signals (loss/accuracy). What's needed are proactive, deeper-level checks during training.

Example silent error: Weight divergence when hugging face was training BLOOM-176B

Related Works

Testing pipeline code, libraries, and compilers

These efforts focus on framework/compiler inconsistencies or isolated error categories, whereas the paper targets a broader set of silent training failures with runtime training invariants.

Monitoring frameworks for training dynamics

These tools require manual and active monitoring, and the metrics are often noisy and can lead to many false alarms when used for detecting silent errors. In contrast, TrainCheck provides an automated solution that captures the precise semantics of training as training invariants and proactively checks them.

Testing trained DL models

These works test the final model weights, not the training process where silent errors arise.

Fault tolerance in DL systems

There are works on elastic scaling, task reallocation, pipeline parallelism, and checkpointing. They improve resilience but do not address silent errors from misconfigurations/bugs/subtle correctness violations.

Invariant mining / rule inference

Prior work mines invariants either at the program-variable level or for distributed protocols/events, which misses the high-level DL-training semantics and generally doesn't transfer beyond the system they were learned from.

Solution Overview

Two-phase workflow

Offline phase (learns rules): automatically infers a set of training invariants from high-quality training pipelines.

Online phase (enforce rules): these invariants are deployed to a given training job to check for violations during training. TrainCheck reports invariant violations with contextual information to assist confirmation and investigation.

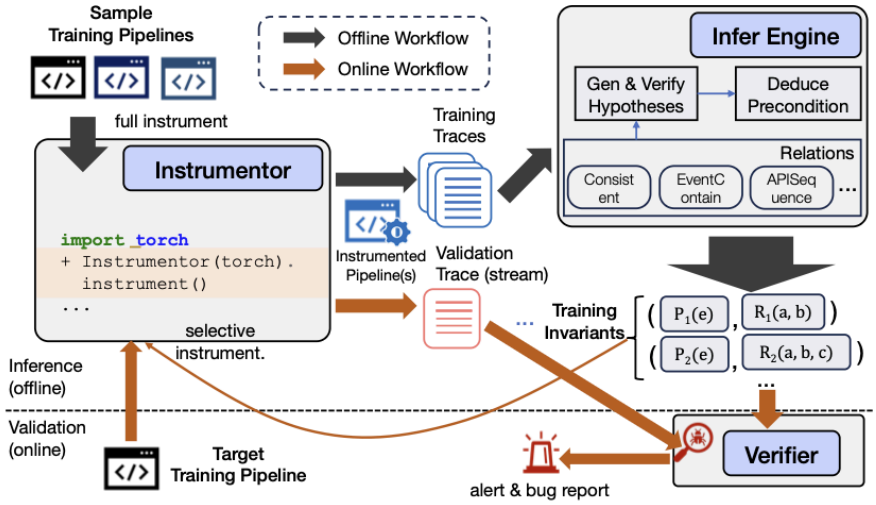


Figure 3: Workflow of TRAINCHECK.

Three major components

1. **Instrumentor**: dynamically instruments DL training programs to collect runtime traces with low overhead
2. **Infer Engine**: analyzes these traces to infer training invariants and their preconditions automatically.
3. **Verifier**: continuously validates a training task against the invariants. When Instrumentor is used in the online stage, it performs selective instrumentation relevant to the inferred invariants.

Relation	Description
$Consistent(V_a, V_b)$	V_a and V_b should have the same values, while the values may change
$EventContain(E_a, E_b)$	E_b must happen in the duration of E_a
$APISequence(I_a, I_b, \ldots)$	I_a, I_b, ... must all occur and in the specified order
$APIArg(I_a, is\_distinct)$	Ensures argument consistency or distinction in all calls to I_a
$APIOutput(I_a, bound\_type)$	The output of I_a must meet certain attribute constraints

Table 2: Sample relations TRAINCHECK defines and supports.

Training invariant

A training invariant is a logical relation over key variables and events, capturing training-specific semantics. Many invariants are conditional, so TrainCheck also infers preconditions.
The above Table 2 shows some sample relations that TrainCheck can support in a training invariant.

Limitations

1. Its instrumentation interferes with JIT compilation tools like torch.compile, preventing the analysis of optimized code paths.
2. TrainCheck is restricted to Python code, limiting its ability to analyze components with significant logic implemented in lower-level languages.

3. Representing tensors in hash form prevents fine-grained numerical analysis, limiting its applicability to detecting instabilities caused by inappropriate hyperparameters. However, this can be complemented by existing research on hyperparameter tuning and numerical defect detection.
4. The cost of invariant inference scales quadratically with respect to input trace size (though it is done offline).

Future Research Directions

1. Systematic debugging support beyond detection.
2. Overhead reduction. "Potential engineering optimizations include asynchronous or batched logging and minimizing redundant instrumentation through static analysis."

Summary of Class Discussion

Q: The invariants are inferred through subsampling. Are they generalizable?
A: There is no guarantee, but many ML pipelines share structural similarity. This allows subsets of inferred invariants to transfer across pipelines (e.g., the use of "zero_grad()").

Q: Where are most bugs detected?
A: Primarily user code, especially in how APIs are used.

Q: In the evaluation, are most of the 18 reported bugs just about the system getting "stuck" (superficial)?
A: Not just getting stuck, but why — uncovering root causes and detecting them proactively.

Q: How does the TrainCheck reduce the trivial preconditions?
A: It filters out trivial failure conditions/relations. [Didn't mention the specifics]

Q: Does the paper explicitly frame invariants in the context of ML, or could the tool apply to other domains?
A: The emphasis is on the transferability of invariants across ML pipelines.

Q: Do similar invariant checking processes exist for other non-ML workloads?
A: Open question…

Q: Why is there little discussion of formal verification work?
A: There is a related paper from the same group with a different approach that uses formal verification.

Q: When training inference engines (with PyTorch sample scripts), what if there's a bad example included?
A: Training samples are picked from known correct workflows like those in the PyTorch docs. In addition, training on multiple samples will allow TrainCheck to only find invariants that are generalizable.

Q: How can the false negative rate be evaluated?
A: The paper does not focus on this metric, but future work could test on bigger and more diverse benchmark data.

Q: How well does TrainCheck generalize to other frameworks?
A: It would rely on the transferability of the ML pipelines. It is a limitation that only PyTorch examples are considered in this work.

Q: Does TrainCheck work well with Just-in-Time compilation?
A: No. This is a limitation of the paper. Due to how the traces are collected, they cannot be done with compiled code.